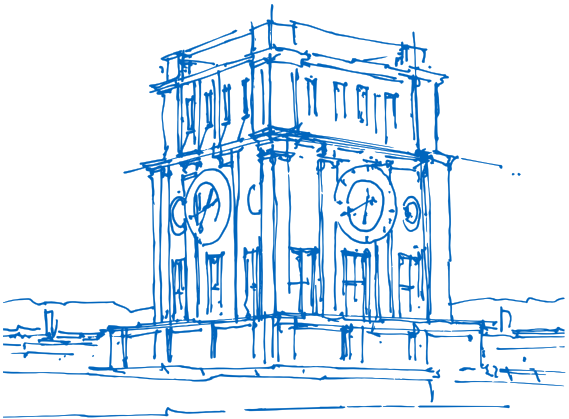
Einführung in die Rechnerarchitektur

Zentralübung #0: Organisatorisches

Dominic Prinz & Jakob Schäffeler
Benjamin Hien & Julian Shen

Lehrstuhl für Rechnerarchitektur & Parallele Systeme
Technische Universität München



Lernkonzept

1. Vorlesung
   - Konzepte und Aufbau von Rechnern
   - Grundlage für das Informatikstudium
2. Zentralübung
   - Praxis zu den Themen der Vorlesung
   - Vertiefung und Anleitung zur Praxis
3. Tutorium
   - praktische Aufgaben mit Übungsblättern und Hausaufgaben
   - Anwendung des Gelernten im interaktiven Umfeld

Wo finde ich was?

1. TUMonline
   - Anmeldung zur Vorlesung, Tutorium und Klausur
2. moodle Kurs (Zugriff automatisch nach Anmeldung in TUMonline)
   - Vorlesungsfolien, Übungsblätter
3. Zulip (Abonnement über den Channel „Kanalgruppen“ 💻)
   - offizielle Kommunikationsplattform für Fragen und Ankündigungen
4. Artemis
   - Hausaufgaben (automatische Korrektur)
5. TUMLive
   - Aufzeichnungen der Vorlesungen und Zentralübungen

Klausur

- Voraussichtlicher Termin: 12.02.2027, 08:00 – 10:00 Uhr
- Wiederholungsklausur voraussichtlich: 01.04.2027, 11:00 – 13:00 Uhr
- Teilnahme an Wiederholungsklausur auch ohne Teilnahme an regulärem Termin möglich
- Nach jetziger Planung alle Hörsäle in Garching
- $\rightarrow$ bereits jetzt in den Kalender schreiben
- Nachteilsausgleiche bitte zunächst zentral beantragen: `https://www.cit.tum.de/cit/studium/studierende/beratung/informatik/barrierefrei-studieren/`
- Anschließend Mail an offizielle ÜL-Mail-Adresse (gesendet von einer `@tum.de` Mail-Adresse): `era@lists.lrz.de`

Tutorien

- Besprechung der Vorlesungsinhalte und Übungsblätter im kleineren Kreis
- Übungsblätter auf Moodle, jeweils ab Freitag 16:00 Uhr
- Teils bewusst etwas überdimensioniert, um auch genügend Material für Selbststudium und Klausurvorbereitung zu bieten
- Ziel: Befähigung typische Aufgaben zum Stoffgebiet zu bearbeiten
- Anmeldung über TUMonline, Matching jede Nacht um 23:59 Uhr, erstes Matching heute
- Alle Räume im MI-Gebäude
- Verfügbare Gruppen auf TUMonline

Hausaufgaben

- Anmeldung zu „Einführung in die Rechnerarchitektur (WS26/27)“[1] über „Course Enrollment“ auf `https://artemis.tum.de/courses/596` ab jetzt
- Normal eine Hausaufgabe pro Woche
- Release am Freitag 16:00, Abgabe Sonntag 23:59 Uhr (10 Tage Zeit)
- 10 Punkte pro regulärer Hausaufgabe
- Notenbonus um eine Notenstufe (0.3/0.4) ab 80 % der erreichbaren Punkte
- Notenbonus nur auf bereits bestandene Klausuren
- Außerdem: unregelmäßige Bonusaufgaben und -punkte

[1] Achtung: nicht mit „Grundlagenpraktikum: Rechnerarchitektur“ verwechseln

Hausaufgaben

- Ziel: Motivation und aktives Arbeiten während des gesamten Semesters, sowie Übung für die Klausur
- Bis zur Deadline unendliche viele Abgabeversuche
- Letzer Versuch zählt für Notenbonus
- Keine Tests nach der Deadline (vorbehaltlich Korrekturen von fehlerhaften Testcases)
- Erste Hausaufgabenabgabe nächsten Sonntag, 25.10.2026, Release **heute**
- Bitte mit Git vertraut machen: `https://rogerdudler.github.io/git-guide/`
- **Keine Teile einer Abgabe in Zulip teilen**

Plagiate

- Die Aufgaben sind in Einzelarbeit zu lösen!
- Wir werden auf Plagiate prüfen und entsprechende Maßnahmen ergreifen.

nach § 22 APSO

- Erster Verstoß → Bewertung mit 5.0U. Prüfung **muss** beim nächsten Versuch bestanden werden, sonst Exmatrikulation
- Wiederholter Verstoß → Exmatrikulation

Plagiate

- Die Aufgaben sind in Einzelarbeit zu lösen!
- Wir werden auf Plagiate prüfen und entsprechende Maßnahmen ergreifen.
- Es ist erlaubt, grundsätzliche Konzepte mit Kommilitonen zu diskutieren.
- Die eigentliche Erstellung des Codes muss jedoch *in Eigenleistung* erfolgen.
- Umsortieren, Umbenennen, usw. stellt keine Eigenleistung dar!

Insbesondere nicht erlaubt

- Einsatz von KI-Tools (z. B. ChatGPT, Claude, Copilot, etc.)
- Übernahme von Code aus Internet oder von Kommilitonen

Zulip

- $\approx$ 570 Studierende, 11 Tutor:innen, 4 Übungsleiter und 2 Professoren
  $\Rightarrow$ skaliert schlecht
- Deshalb bitte diese Schritte befolgen, wenn eine Frage auftritt:
  1. Folien durchsuchen (STRG+F) und Suchmaschine der Wahl befragen
  2. Frage, je nach Möglichkeit, im Tutorium fragen
  3. Frage auf Zulip suchen! (Fragen werden nicht doppelt beantwortet)
  4. Frage **im richtigen Channel** stellen
- Nur so können wir garantieren, dass jede Frage beantwortet wird :)

Bitte …

- … inhaltliche Fragen nicht privat an TutorInnen stellen
- … immer den richtigen Channel benutzen
- … wenn nötig, ein neues Thema erstellen
- … Fragen nicht in ein bereits als gelöst markiertes Thema stellen
- … keine Screenshots benutzen, lieber die Fehlermeldung kopieren
- … ausführlich (mit Infos, was bereits versucht wurde) fragen. Nicht nur „Das geht nicht.“
- … niemanden mit @ pingen
- … Fragen zu Hausaufgaben in den Channel des jeweiligen Übungsblatts stellen
- … beachten, dass wir sonst Nachrichten löschen werden
- … trotzdem keine Angst haben, Fragen zu stellen… einfach Mühe beim Fragestellen geben – dann freuen wir uns alle

Machen Sie sich ein Lesezeichen…

… denn diese Folien beantworten die meisten organisatorischen Fragen

Inhaltsübersicht – 1. Teil: Assembler Programmierung

- maschinennahe Programmiersprache mit voller Kontrolle über den Prozessor, Speicher, Register
- 1:1 Übersetzung in Maschinencode
- Assembler-Code ist spezifisch für eine bestimmte Prozessorarchitektur (bei uns RISC-V)
- Ziel: Assembler Programme schreiben können mit . . .
  - . . . Unterprogrammen
  - . . . Sprüngen und Schleifen
  - . . . Speicherzugriffen
  - . . . Rekursion
  - . . . Calling Convention

Programm in Hochsprache:

```
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}
```

Programm in RISC-V:

```
factorial(int):
        mv      a5,a0
        li      a0,1
        mv      a3,a0
        bleu    a5,a0,.L1
.L2:
        mv      a4,a5
        addi    a5,a5,-1
        mul     a0,a0,a4
        bne     a5,a3,.L2
.L1:
        ret
```

Inhaltsübersicht – 2. Teil: RISC-V Prozessor

- Bauen eines RISV-C Prozessors aus elemenaren Gattern
- Umsetzung von Assembler-Befehle auf Prozessorhardware
- Optimierung von Schaltungen für höhere Effizienz
- Ziel:
  - Verständnis wie RISC-V Prozessor funktioniert
  - Selbstständig neue Befehle in Prozessor implementieren können

